JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class EnumMap<K extends Enum<K>, V>

java.lang.Object
    java.util.AbstractMap<K,V>
        java.util.EnumMap<K,V>

Type Parameters:

K - the enum type of keys maintained by this map

V - the type of mapped values

All Implemented Interfaces:

Serializable, Cloneable, Map<K,V>

```
public class EnumMap<K extends Enum<K>, V>
extends AbstractMap<K,V>
implements Serializable, Cloneable
```

A specialized Map implementation for use with enum type keys. All of the keys in an enum map must come from a single enum type that is specified, explicitly or implicitly, when the map is created. Enum maps are represented internally as arrays. This representation is extremely compact and efficient.

Enum maps are maintained in the *natural order* of their keys (the order in which the enum constants are declared). This is reflected in the iterators returned by the collections views (keySet(), entrySet(), and values()).

Iterators returned by the collection views are *weakly consistent*: they will never throw ConcurrentModificationException and they may or may not show the effects of any modifications to the map that occur while the iteration is in progress.

Null keys are not permitted. Attempts to insert a null key will throw NullPointerException. Attempts to test for the presence of a null key or to remove one will, however, function properly. Null values are permitted.

Like most collection implementations EnumMap is not synchronized. If multiple threads access an enum map concurrently, and at least one of the threads modifies the map, it should be synchronized externally. This is typically accomplished by synchronizing on some object that naturally encapsulates the enum map. If no such object exists, the map should be "wrapped" using the Collections.synchronizedMap(Map) method. This is best done at creation time, to prevent accidental unsynchronized access:

```
    Map<EnumKey, V> m
        = Collections.synchronizedMap(new EnumMap<EnumKey, V>(...));
```

Implementation note: All basic operations execute in constant time. They are likely (though not guaranteed) to be faster than their `HashMap` counterparts.

This class is a member of the Java Collections Framework.

Since:

1.5

See Also:

`EnumSet`, Serialized Form

Nested Class Summary

Nested classes/interfaces declared in class AbstractMap
`AbstractMap.SimpleEntry<K,V>`, `AbstractMap.SimpleImmutableEntry<K,V>`

Nested classes/interfaces declared in interface Map
`Map.Entry<K,V>`

Constructor Summary

Constructors

Constructor	Description
`EnumMap(Class<K> keyType)`	Creates an empty enum map with the specified key type.
`EnumMap(EnumMap<K, ? extends V> m)`	Creates an enum map with the same key type as the specified enum map, initially containing the same mappings (if any).
`EnumMap(Map<K, ? extends V> m)`	Creates an enum map initialized from the specified map.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`void`	`clear()`	Removes all mappings from this map.

EnumMap<K,V>	**clone**()	Returns a shallow copy of this enum map.
boolean	**containsKey**(Object key)	Returns `true` if this map contains a mapping for the specified key.
boolean	**containsValue** (Object value)	Returns `true` if this map maps one or more keys to the specified value.
Set<Map.Entry<K,V>>	**entrySet**()	Returns a `Set` view of the mappings contained in this map.
boolean	**equals**(Object o)	Compares the specified object with this map for equality.
V	**get**(Object key)	Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.
int	**hashCode**()	Returns the hash code value for this map.
Set<K>	**keySet**()	Returns a `Set` view of the keys contained in this map.
V	**put**(K key, V value)	Associates the specified value with the specified key in this map.
void	**putAll**(Map<? extends K, ? extends V> m)	Copies all of the mappings from the specified map to this map.
V	**remove**(Object key)	Removes the mapping for this key from this map if present.
int	**size**()	

		Returns the number of key-value mappings in this map.
Collection<V>	**values**()	Returns a Collection view of the values contained in this map.

Methods declared in class AbstractMap

isEmpty, toString

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface Map

compute, computeIfAbsent, computeIfPresent, forEach, getOrDefault, merge, putIfAbsent, remove, replace, replace, replaceAll

Constructor Details

EnumMap

```
public EnumMap(Class<K> keyType)
```

Creates an empty enum map with the specified key type.

Parameters:

keyType - the class object of the key type for this enum map

Throws:

NullPointerException - if keyType is null

EnumMap

```
public EnumMap(EnumMap<K, ? extends V> m)
```

Creates an enum map with the same key type as the specified enum map, initially containing the same mappings (if any).

Parameters:

m - the enum map from which to initialize this enum map

Throws:

NullPointerException - if m is null

EnumMap

```
public EnumMap(Map<K, ? extends V> m)
```

Creates an enum map initialized from the specified map. If the specified map is an `EnumMap` instance, this constructor behaves identically to `EnumMap(EnumMap)`. Otherwise, the specified map must contain at least one mapping (in order to determine the new enum map's key type).

Parameters:

`m` - the map from which to initialize this enum map

Throws:

`IllegalArgumentException` - if `m` is not an `EnumMap` instance and contains no mappings

`NullPointerException` - if `m` is null

Method Details

size

```
public int size()
```

Returns the number of key-value mappings in this map.

Specified by:

`size` in interface `Map<K extends Enum<K>, V>`

Overrides:

`size` in class `AbstractMap<K extends Enum<K>, V>`

Returns:

the number of key-value mappings in this map

containsValue

```
public boolean containsValue(Object value)
```

Returns `true` if this map maps one or more keys to the specified value.

Specified by:

`containsValue` in interface `Map<K extends Enum<K>, V>`

Overrides:

`containsValue` in class `AbstractMap<K extends Enum<K>, V>`

Parameters:

`value` - the value whose presence in this map is to be tested

Returns:

`true` if this map maps one or more keys to this value

containsKey

```
public boolean containsKey(Object key)
```

Returns `true` if this map contains a mapping for the specified key.

Specified by:

`containsKey` in interface `Map<K extends Enum<K>, V>`

Overrides:

`containsKey` in class `AbstractMap<K extends Enum<K>, V>`

Parameters:

`key` - the key whose presence in this map is to be tested

Returns:

`true` if this map contains a mapping for the specified key

get

```
public V get(Object key)
```

Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.

More formally, if this map contains a mapping from a key `k` to a value `v` such that `(key == k)`, then this method returns `v`; otherwise it returns `null`. (There can be at most one such mapping.)

A return value of `null` does not *necessarily* indicate that the map contains no mapping for the key; it's also possible that the map explicitly maps the key to `null`. The `containsKey` operation may be used to distinguish these two cases.

Specified by:

`get` in interface `Map<K extends Enum<K>, V>`

Overrides:

`get` in class `AbstractMap<K extends Enum<K>, V>`

Parameters:

`key` - the key whose associated value is to be returned

Returns:

the value to which the specified key is mapped, or `null` if this map contains no mapping for the key

put

```
public V put(K key,
             V value)
```

Associates the specified value with the specified key in this map. If the map previously contained a mapping for this key, the old value is replaced.

Specified by:

`put` in interface `Map<K extends Enum<K>, V>`

Overrides:

`put` in class `AbstractMap<K extends Enum<K>, V>`

Parameters:

`key` - the key with which the specified value is to be associated

`value` - the value to be associated with the specified key

Returns:

the previous value associated with specified key, or `null` if there was no mapping for key. (A `null` return can also indicate that the map previously associated `null` with the specified key.)

Throws:

`NullPointerException` - if the specified key is null

remove

```
public V remove(Object key)
```

Removes the mapping for this key from this map if present.

Specified by:

`remove` in interface `Map<K extends Enum<K>, V>`

Overrides:

`remove` in class `AbstractMap<K extends Enum<K>, V>`

Parameters:

`key` - the key whose mapping is to be removed from the map

Returns:

the previous value associated with specified key, or `null` if there was no entry for key. (A `null` return can also indicate that the map previously associated `null` with the specified key.)

putAll

```
public void putAll(Map<? extends K, ? extends V> m)
```

Copies all of the mappings from the specified map to this map. These mappings will replace any mappings that this map had for any of the keys currently in the specified map.

Specified by:

`putAll` in interface `Map<K extends Enum<K>, V>`

Overrides:

`putAll` in class `AbstractMap<K extends Enum<K>, V>`

Parameters:

`m` - the mappings to be stored in this map

Throws:

`NullPointerException` - the specified map is null, or if one or more keys in the specified map are null

clear

```
public void clear()
```

Removes all mappings from this map.

Specified by:

`clear` in interface `Map<K extends Enum<K>, V>`

Overrides:

`clear` in class `AbstractMap<K extends Enum<K>, V>`

keySet

```
public Set<K> keySet()
```

Returns a `Set` view of the keys contained in this map. The returned set obeys the general contract outlined in `Map.keySet()`. The set's iterator will return the keys in their natural order (the order in which the enum constants are declared).

Specified by:

`keySet` in interface `Map<K extends Enum<K>, V>`

Overrides:

`keySet` in class `AbstractMap<K extends Enum<K>, V>`

Returns:

a set view of the keys contained in this enum map

values

```
public Collection<V> values()
```

Returns a `Collection` view of the values contained in this map. The returned collection obeys the general contract outlined in `Map.values()`. The collection's iterator will return the values in the order their corresponding keys appear in map, which is their natural order (the order in which the enum constants are declared).

Specified by:

`values` in interface `Map<K extends Enum<K>, V>`

Overrides:

`values` in class `AbstractMap<K extends Enum<K>, V>`

Returns:

a collection view of the values contained in this map

entrySet

```
public Set<Map.Entry<K,V>> entrySet()
```

Returns a `Set` view of the mappings contained in this map. The returned set obeys the general contract outlined in `Map.keySet()`. The set's iterator will return the mappings in the order their keys appear in map, which is their natural order (the order in which the enum constants are declared).

Specified by:

`entrySet` in interface `Map<K extends Enum<K>, V>`

Returns:

a set view of the mappings contained in this enum map

equals

```
public boolean equals(Object o)
```

Compares the specified object with this map for equality. Returns `true` if the given object is also a map and the two maps represent the same mappings, as specified in the `Map.equals(Object)` contract.

Specified by:

`equals` in interface `Map<K extends Enum<K>, V>`

Overrides:

`equals` in class `AbstractMap<K extends Enum<K>, V>`

Parameters:

`o` - the object to be compared for equality with this map

Returns:

`true` if the specified object is equal to this map

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns the hash code value for this map. The hash code of a map is defined to be the sum of the hash codes of each entry in the map.

Specified by:

`hashCode` in interface `Map<K extends Enum<K>, V>`

Overrides:

`hashCode` in class `AbstractMap<K extends Enum<K>, V>`

Returns:

the hash code value for this map

See Also:

`Map.Entry.hashCode()`, `Object.equals(Object)`, `Set.equals(Object)`

clone

```
public EnumMap<K,V> clone()
```

Returns a shallow copy of this enum map. The values themselves are not cloned.

Overrides:

`clone` in class `AbstractMap<K extends Enum<K>, V>`

Returns:

a shallow copy of this enum map

See Also:

`Cloneable`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.